

پرفورمنس، امنیت، سرور و نقشه راه

پاسخ چهار سؤال شما، بر پایه‌ی استانداردهای رایج (OWASP و فهرست Node.js Best Practices) و متناسب با همین پروژه - نه یک چک‌لیست عمومی. هر بندی که برای این سامانه لازم نیست، عمداً حذف شده و دلیلش نوشته شده.

۱ پرفورمنس - هدف ۱۰,۰۰۰ کاربر همزمان

اول یک تصحیح مهم در صورت مسئله

«۱۰,۰۰۰ کاربر همزمان» با «۱۰,۰۰۰ درخواست در ثانیه» یکی نیست. یک نماینده هر ۱۵ تا ۳۰ ثانیه یک کلیک می‌کند. پس ۱۰,۰۰۰ کاربر فعال یعنی حدود

۳۰۰ تا ۷۰۰ درخواست در ثانیه

- عددی که یک سرور معمولی با معماری درست جواب می‌دهد. مشکل معمولاً از تعداد کاربر نمی‌آید، از

چند تصمیم غلط در کد و دیتابیس

می‌آید که در ادامه آمده.

۱.۱ مقیاس واقعی در برابر مقیاس هدف

سناریو	تعداد حساب	کاربر همزمان اوج	نتیجه
سال اول (۵۰ نماینده‌ی مادر)	۲,۵۵۰~	۱۵۰ تا ۴۰۰	یک سرور، بدون هیچ پیچیدگی
رشد چند برابری	۱۰,۰۰۰~	۸۰۰ تا ۱,۵۰۰	همان سرور، فقط قوی‌تر
هدف شما	-	۱۰,۰۰۰	۲ تا ۳ نمونه‌ی اپ + دیتابیس جدا

پروژه را طوری می‌نویسم که گذر از حالت اول به سوم فقط تغییر تنظیمات باشد، نه بازنویسی کد. این تفاوت اصلی معماری درست و غلط است.

۱.۲ نه تصمیمی که واقعاً پرفورمنس را تعیین می‌کنند

#	موضوع	کاری که می‌کنیم و چرا
۱	جستجوی متن فارسی	بزرگ‌ترین ریسک این پروژه همین است. جستجوی ساده روی متن ('LIKE %') با ۱۰۰ هزار حواله کند می‌شود چون کل جدول را می‌خواند. از ایندکس <code>pg_trgm</code> پستگرس استفاده می‌کنیم که همین جستجو را روی میلیون‌ها ردیف هم زیر ۵۰ میلی‌ثانیه نگه می‌دارد.
۲	صفحه‌بندی	روش رایج <code>OFFSET 10000</code> است که هر بار ۱۰ هزار ردیف را می‌خواند و دور می‌ریزد. به‌جایش صفحه‌بندی کلیدی می‌گذاریم: «از این شناسه به بعد». صفحه‌ی هزارم به همان سرعت صفحه‌ی اول باز می‌شود.
۳	مشکل N+1	اگر لیست ۲۰ حواله‌ای برای هر ردیف یک کوئری جدا برای مالکش بزنند، می‌شود ۲۱ کوئری به‌جای ۱. با <code>include</code> در یک کوئری می‌گیریم. این تک‌تفاوت، بار دیتابیس را چند برابر کم می‌کند.
۴	استخر اتصال دیتابیس	پستگرس پیش‌فرض حدود ۱۰۰ اتصال قبول می‌کند. زیر بار، اپ می‌خواهد صدها اتصال باز کند و همه‌چیز می‌خوابد — شایع‌ترین علت سقوط این نوع اپ‌ها . جلوییش را با <code>PgBouncer</code> در حالت <code>transaction</code> و سقف اتصال روی <code>Prisma</code> می‌گیریم.
۵	شمارنده‌ی سقف نمایش	اگر برای هر بار نمایش مشخصات، یک <code>COUNT</code> روی جدول بزنیم، این جدول با میلیون‌ها ردیف کند می‌شود. شمارنده‌ها را در <code>Redis</code> نگه می‌داریم که در حد میکروتانیه جواب می‌دهد و اتمیک هم هست (دو کلیک همزمان، دوتا حساب می‌شود نه یکی).
۶	بررسی نشست	تک‌نشستی یعنی هر درخواست باید نشست را بررسی کند. اگر هر بار به پستگرس بزنیم، فقط همین یک کار می‌شود سنگین‌ترین بار سامانه. نشست‌ها در <code>Redis</code> نگه داشته می‌شوند.
۷	کش صفحه‌ی استعلام	یک ویژگی طراحی به کمکمان می‌آید: چون مشخصات تماس برای همه مخفی است ، لیست حواله‌ها برای همه‌ی کاربران یکسان است. پس می‌شود همان یک نتیجه را ۱۰ تا ۳۰ ثانیه کش کرد و به همه داد. فقط بخش «باز شده برای من» شخصی است و جدا می‌آید.
۸	جدول‌های پر حجم	<code>ActivityLog</code> و <code>ContactReveal</code> سریع بزرگ می‌شوند. ماهانه پارتیشن می‌شوند تا کوئری‌های روزمره فقط پارتیشن جاری را بخوانند و جدول قدیمی سنگینشان نکند.
۹	کارهای پس‌زمینه	بستن خودکار آگهی‌ها، ارسال پیامک و صدور صورتحساب در یک پردازهی جدا اجرا می‌شوند. اگر پتل پیامک کند باشد یا جواب ندهد، سایت نباید کند شود.

سنجش، نه حدس

قبل از تحویل، با ابزار `k6` تست بار می‌گیریم: ۱,۰۰۰ کاربر مجازی روی سنگین‌ترین صفحه (استعلام با فیلتر). عدد واقعی را می‌بینیم، نه ادعا. اگر جایی کند بود، همان‌جا اصلاح می‌شود. هدف:

زیر ۳۰۰ میلی‌ثانیه

برای صفحه‌ی استعلام و

زیر ۱۰۰ میلی‌ثانیه

برای بقیه.

دارایی اصلی این سامانه، شماره تلفن‌هاست

بیشتر حمله‌ها به این سامانه برای خراب کردنش نیست، برای

بیرون کشیدن دیتای تماس

است. بنابراین علاوه بر ضدحمله‌های استاندارد، کنترل‌های خاص افشای دیتا هم گذاشته شده — بند ۲.۳.

۲.۱ چهار موردی که اسم بردید

تهدید	راهکار
XSS تزریق اسکریپت	React همه‌ی متن‌ها را خودکار خنثی می‌کند؛ قانون پروژه: <code>dangerouslySetInnerHTML</code> ممنوع. به‌علاوه هدر CSP که حتی اگر اسکریپتی تزریق شد، مرورگر اجزایش نکند. همه‌ی ورودی‌ها با Zod اعتبارسنجی می‌شوند — هم شکل، هم طول، هم بازه. نکته‌ی کمترشناخته: خروجی اکسل هم راه تزریق است. سلولی که با <code>=</code> شروع شود در اکسل کاربر اجرا می‌شود؛ خنثی می‌شود.
SQL Injection	با Prisma همه‌ی کوئری‌ها پارامتری‌اند و رشته‌ی SQL دستی ساخته نمی‌شود. قانون پروژه: <code>\$queryRawUnsafe</code> ممنوع، و اگر جایی SQL خام لازم شد فقط با پارامتر.
DDoS	صادقانه: حمله‌ی حجمی را نمی‌شود در کد دفع کرد — قبل از رسیدن به سرور باید متوقف شود. دو لایه: (۱) سرویس ابری جلوی سایت مثل ابر آروان یا کلادفلر که ترافیک را فیلتر کند؛ (۲) در خود اپ، محدودیت نرخ درخواست روی Redis، سقف اندازه‌ی درخواست، و قفل ۱۵ دقیقه‌ای بعد از ۵ ورود ناموفق.
لو رفتن توکن	توکن نشست اصلاً در دسترس جاوااسکریپت نیست: کوکی با <code>HttpOnly + Secure + SameSite=Strict</code> و پیشوند <code>__Host-</code> . در دیتابیس فقط هش توکن نگه داشته می‌شود، پس حتی با دسترسی به دیتابیس هم توکن قابل استفاده نیست. توکن بعد از ورود موفق چرخانده می‌شود، و تگ‌نشستی خودش یک کنترل امنیتی است: توکن دزدیده‌شده با اولین ورود واقعی کاربر باطل می‌شود.

۲.۲ مواردی که اسم نبردید ولی لازم‌اند

مورد	کار	اولویت
CSRF	کوکی SameSite=Strict به‌علاوه توکن CSRF روی فرم‌های تغییردهنده	الزامی
رمز عبور	هش با bcrypt (یا argon2)؛ رمز خام هرگز ذخیره یا لاگ نمی‌شود	الزامی
هدرهای امنیتی	HSTS، جلوگیری از قاب‌شدن سایت در iframe، منع حدس نوع فایل	الزامی
مدیریت رمزها و کلیدها	کلید API پیامک و رمز دیتابیس فقط در متغیر محیطی؛ هرگز داخل کد یا گیت	الزامی
پیام خطا	خطای فنی به کاربر نشان داده نمی‌شود؛ فقط شناسه‌ی خطا. جزئیات در لاگ سرور	الزامی
لاگ تمیز	رمز، توکن و کد پیامکی هرگز در لاگ نوشته نمی‌شوند	الزامی
پیوست تیکت	محدودیت نوع و حجم، ذخیره بیرون از مسیر عمومی، نام تصادفی	الزامی
اجرا با کاربر غیر روت	سرویس با کاربر کم‌دسترسی اجرا شود	الزامی
بررسی وابستگی‌ها	npm audit در CI و به‌روزرسانی منظم	الزامی
بکاپ رمزگذاری‌شده	روزانه، روی مقصد جدا، و تست بازیابی – بکاپ تست‌نشده یعنی بکاپ ندارید	الزامی
احراز هویت دومرحله‌ای پنل مدیریت	حساب‌های خودمان هم کد پیامکی داشته باشند	بعد از لانچ
تست نفوذ	یک بررسی بیرونی وقتی سامانه جا افتاد	بعد از لانچ

۲.۳ مهم‌ترین بند این سند

مخفی‌سازی باید سمت سرور باشد، نه سمت نمایش

اگر سرور شماره‌ی تماس را در پاسخ بفرستد و فقط رابط کاربری آن را نشان ندهد، هر کسی با ابزار توسعه‌دهنده‌ی مرورگر – که با یک کلید F12 باز می‌شود –

کل شماره‌ها را می‌بیند

؛ بدون اینکه سقفی مصرف شود و بدون اینکه چیزی در لاگ ثبت گردد. یعنی هم مدل درآمدی و هم مانیتورینگ یکجا دور زده می‌شود.

قانون پروژه:

شماره‌ی تماس فقط وقتی وارد پاسخ سرور می‌شود که آن نماینده واقعاً دکمه را زده و در دیتابیس ثبت شده باشد.

در بقیه‌ی موارد اصلاً از دیتابیس خوانده نمی‌شود. این را جزو تست‌های خودکار می‌گذارم تا اگر روزی کسی خرابش کرد، تست قرمز شود.

- خروجی اکسل فقط برای مدیر کل، و هر بار گرفتن خروجی خودش در لاگ ثبت می‌شود.
- سقف روزانه و دوره‌ای، به‌علاوه علامت‌گذاری رفتار غیرعادی در مانیتورینگ.
- هیچ صفحه‌ای بدون ورود قابل دیدن نیست؛ سامانه موتور جستجو را هم راه نمی‌دهد.

۳.۱ مشخصات

مرحله	پردازنده	رم	دیسک	توضیح
شروع (پیشنهاد من)	۴ هسته	۸ گیگ	۱۰۰ گیگ NVMe	یک سرور، همه چیز رویش: اپ + پستگرس + Redis. برای سال اول با فاصله کافی است.
رشد	۸ هسته	۱۶ گیگ	۲۰۰ گیگ NVMe	همان یک سرور، فقط ارتقا. تا چند هزار کاربر فعال جواب می‌دهد.
هدف ۱۰ هزار	۲ سرور	۱۶ + ۸ گیگ	—	دیتابیس روی سرور جدا، اپ روی سرور دیگر (و در صورت نیاز دو نمونه پشت یک توزیع‌کننده‌ی بار).

از سرور کوچک شروع کنید

خرید سرور بزرگ از روز اول، پول دور ریختن است. چون معماری از ابتدا درست نوشته می‌شود، ارتقا در آینده چند ساعت کار است نه چند هفته. ۴ هسته و ۸ گیگ شروع کنید و بر اساس عدد واقعی مصرف تصمیم بگیرید.

۳.۲ چیزهایی که باید تهیه کنید

مورد	توضیح
سرور ابری ایران	ابر آروان، پارس‌پک، ایران‌سرور و مشابه. سرور ایران انتخاب شد چون کاربران داخل ایران‌اند و پنل پیامک هم داخلی است. قیمت‌ها را موقع خرید بگیرید؛ در این بازار سریع عوض می‌شوند.
سرویس ابری جلوی سایت	برای CDN و سپر DDoS. ابر آروان گزینه‌ی متداول داخلی است.
دامنه	یک <code>.ir</code> و ترجیحاً <code>.com</code> . همان اسم — بند ۴.
گواهی SSL	رایگان با Let's Encrypt و تمدید خودکار.
پنل پیامک	کاوه‌نگار، ملی پیامک یا مشابه، به‌علاوه یک خط خدماتی. برای کد ورود و هشدارها لازم است.
فضای بکاپ	یک فضای ذخیره‌سازی جدا از سرور اصلی. بکاپی که روی همان سرور باشد، با از دست رفتن سرور از بین می‌رود.

۴ اسم پروژه

معیار انتخاب

اسم باید هم «حواله» (فاز ۱) و هم «فروش خودرو صفر» (فاز ۲) را در خود جا بدهد. برای همین اسم‌هایی مثل «حواله‌بازار» را کنار گذاشتم — فاز دو در آن جا نمی‌شود و مجبورید برند عوض کنید. اسم باید کوتاه، تلفظ‌راحت، و از برندهای موجود بازار خودرو (باما، دیوار، همراه‌مکانیک) به‌قدر کافی دور باشد.

#	اسم	لاتین	چرا
۱	خودروبان	khodroban	پیشنهاد اول من. «بان» یعنی متولی و نگهبان – دقیقاً کاری که می‌کنید: واسطه‌ی مطمئن. هم حواله را می‌گیرد هم خودرو صفر را. فارسی، آشنا، و جای برند شدن دارد.
۲	آتیه خودرو	atiyekhodro	«آتیه» یعنی آینده، و حواله دقیقاً خودروی آینده است. برای فاز ۲ هم بی‌مشکل است. لحن رسمی و قابل اعتماد دارد که برای معامله‌های میلیاردی خوب است.
۳	کاربان	karban	کوتاه‌ترین گزینه، خوش‌تلفظ در فارسی و انگلیسی. مزیتش برای رشد آینده است، عیبش این که به‌تنهایی نمی‌گوید کارتان خودرو است.
۴	پلاک	pelak	یک کلمه، کاملاً مرتبط با خودرو، خیلی راحت در ذهن می‌ماند. احتمال اینکه دامنه‌اش آزاد باشد کم است – ولی ارزش بررسی دارد.
۵	وصال خودرو	vesalkhodro	«وصال» یعنی به هم رساندن – همان کاری که سامانه می‌کند: رساندن حواله‌دار به خریدار.
۶	فردا خودرو	fardakhodro	ساده‌ترین و صمیمی‌ترین گزینه. همان مفهوم «آتیه» با زبان روزمره‌تر.
۷	رهام خودرو	rahamkhodro	اسم خنثی و رسمی. اگر می‌خواهید اسم شرکتی باشد نه توصیفی، این گزینه است.
۸	خودروکار	khodrokar	صریح و کاربردی، لحن «ابزار کار حرفه‌ای‌ها» دارد که با مخاطب شما (نمایندگی‌ها، نه مصرف‌کننده) می‌خواند.

قبل از انتخاب نهایی حتماً چهار چیز را چک کنید

آزاد بودن دامنه‌ی **.ir** و **.com** · آزاد بودن آی‌دی اینستاگرام و تلگرام · قابل ثبت بودن به‌عنوان برند · و اینکه با اسم شرکت فعالی در همین حوزه اشتباه گرفته نشود.

من به اینترنت لحظه‌ای دسترسی ندارم که آزاد بودن دامنه‌ها را ببینم؛ این بررسی باید دستی انجام شود.

درباره‌ی زمان‌ها

این‌ها برآورد است، نه قول. اگر وسط کار تصمیم محصولی عوض شود یا بازخورد بچه‌ها تغییرات بزرگ بیاورد، جابه‌جا می‌شود. ترتیب گام‌ها اما مهم است: هر گام روی گام قبل سوار می‌شود.

گام	کار	با کیست	برآورد	خروجی
۰	بازخورد عرفان و مهدی	شما	—	لیست ادیت‌ها روی موک‌آپ و سند
۰	تهیه‌ی زیرساخت	شما	موازی	اسم و دامنه، سرور، پنل پیامک و خط خدماتی
۱	اسکلت پروژه، ورود، نقش‌ها، تک‌نشستی	من	۱~ هفته	می‌شود وارد شد و نقش‌ها کار می‌کنند
۲	حواله: ثبت، استعمال، فیلتر، مخفی‌سازی و سقف	من	۱/۵~ هفته	قلب سامانه کار می‌کند
۳	اشتراک، حالت مازول، صورتحساب ظرفیت	من	۱~ هفته	مدل درآمدی برقرار است
۴	گزارش تخلف و تیکت پشتیبانی	من	۱~ هفته	سلامت بازار و پشتیبانی
۵	پنل مدیریت و مانیتورینگ	من	۱~ هفته	شما کنترل کامل دارید
۶	سخت‌سازی امنیتی، تست بار، بکاپ، راه‌اندازی سرور	من	۱~ هفته	آماده‌ی کاربر واقعی
۷	پایلوت با ۵ تا ۱۰ نمایندگی واقعی	هر دو	۲~ هفته	اشکالات واقعی، نه فرضی
۸	اصلاح بازخورد پایلوت و لانچ	من	۱~ هفته	لانچ

جمع تقریبی از شروع کدنویسی تا لانچ: ۹ تا ۱۰ هفته، که ۲ هفته‌اش پایلوت با کاربر واقعی است.

۵.۱ گام ۷ را جدی بگیرید

پایلوت مهم‌ترین گام این جدول است

ما سه دور روی کاغذ بازبینی کردیم و ۳۰ اصلاح درآمد. ولی چیزهایی هست که فقط وقتی معلوم می‌شود که یک نماینده‌ی واقعی با یک مشتری واقعی پشت خط، بخواهد کار کند. با ۵ تا ۱۰ نمایندگی که به آن‌ها دسترسی می‌دهید، مشکلات را قبل از اینکه ۵۰ نماینده‌ی پولی ببینند پیدا می‌کنیم. پیشنهادم: در دوره‌ی پایلوت اشتراک رایگان بدهید و در عوض بازخورد بگیرید.

۵.۲ چکلیست «قبل از لانچ»

مورد	چرا
بکاپ روزانه فعال و یکبار تست بازیابی شده	کل دارایی این کسب‌وکار همین دیتاست
پنل پیامک وصل و کد دو عاملی روشن	تا آن روز ورود فقط با رمز است
سپر DDOS و SSL فعال	روز اول لانچ بدترین زمان برای یادگیری این‌هاست
تست بار گرفته شده و اعداد ثبت شده	تا بعداً بدانیم کند شدن از کجاست
تست خودکار «شماره در پاسخ سرور نیاید»	بند ۲.۳ – مهم‌ترین کنترل سامانه
قوانین و شرایط استفاده نوشته شده	برای تعلیق حساب و برخورد با تخلف، پشتوانه لازم دارید
یک حساب مدیر کل با رمز قوی، بقیه پشتیبانی/مالی	حساب مدیر کل را روزمره استفاده نکنید

۵.۳ آماده‌سازی برای فاز ۲ (فروش خودرو صفر)

- مدل داده از الان چندشرکتی است – اضافه کردن شرکت خودروساز جدید فقط یک ردیف در تنظیمات است.
- ساختار «آگهی» طوری نوشته می‌شود که نوع سومی جز حواله‌ی فروش و درخواست خرید بتواند اضافه شود.
- نقش‌ها و اشتراک از الان جدا از حواله‌اند، پس فاز ۲ لازم نیست دوباره ساخته شود.
- **ولی وعده‌ی بیشتری نمی‌دهم:** فاز ۲ منطق خودش را دارد و وقتی تعریفش کردید، طراحی جدا می‌خواهد. کاری که الان می‌کنم این است که راه را نبندم، نه اینکه فاز ۲ را از قبل بسازم.